
NEURAL NETWORKS

MACHINE LEARNING LECTURE NOTES

Yue Ning

Stevens Institute of Technology
yue.ning@stevens.edu

1 Review of Supervised Learning

Supervised learning involves learning a function from labeled examples—pairs $(\mathbf{x}_n, y_n)$, where the goal is to infer a mapping $f(\mathbf{x}; \mathbf{w})$ that generalizes well to unseen data.

We want to find parameters $(\mathbf{w})$ such that the predicted output $(\hat{y})$ is close to the true label (y) . This is achieved by minimizing a loss function over all training samples:

$$\min_{\mathbf{w} \in \mathbb{R}^d} \mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{x}_n, y_n; \mathbf{w})$$

Typical supervised tasks include:

- **Classification:** Examples are email spam classification, cancer detection, image recognition. We use $y_n \in [0, 1]$ to denote the label for binary classification. In multi-class classification, we often use a one-hot vector to denote the label: $y_n \in \mathbb{B}^K$.
- **Regression:** Examples are predicting house prices, temperature forecasting.

A summary of these two tasks are presented in Table 1

Table 1: Summary of Classification and Regression

Task	Output Type	Relationship	Loss Function	Optimization
Classification	Discrete y	Margin (score, y)	Logistic, hinge, cross-entropy	Gradient Descent (GD)
Regression	Continuous y	Residual (score, y)	Squared error, absolute deviation	Closed-form or GD

2 What is a Neural Network?

A **neural network** is best viewed as a **mathematical function** that transforms inputs into outputs via a sequence of linear transformations and nonlinear activations. Although inspired by biological neurons, the computational neural network is an abstract mathematical model designed for function approximation and pattern recognition [2]. Neural networks have been successfully applied in computer vision, speech recognition, gaming, and many other domains. They can capture complex, nonlinear relationships and perform well given large datasets. They can provide fast predictions once trained.

2.1 Motivation: Beyond Linear Models

A single-layer perceptron can represent only linearly separable problems (e.g., AND, OR). It fails for problems like **XOR**, which are non-linearly separable.

Neural networks with hidden layers overcome this limitation:

- Hidden layers learn intermediate representations of the input.
- Nonlinear activations allow approximation of any function (universal approximation theorem).



Figure 1: Predicting car collisions in one-dimensional space (source)

This is why neural networks fundamentally extend the capabilities of classical linear models. We use a toy example to explain decomposition of a problem and function of hidden neurons. Assuming that we predict if two cars will have a collision in one-dimensional space. As shown in Figure 1, we consider inputs $x = [x_1, x_2]^T$, where x_1, x_2 are the positions of two cars. We will predict if there is a car collision given the positions of two cars. The true function for safety is:

$$y = \text{sign}(|x_1 - x_2| - 1)$$

We can decompose the logic into hidden components:

- Hidden neuron h_1 : car 1 is more than 1 unit ahead of car 2 which means $h_1 = I(x_1 - x_2 \geq 1)$
- Hidden neuron h_2 : car 2 is more than 1 unit ahead of car 1 which means $h_2 = I(x_2 - x_1 \geq 1)$

These binary checks capture separate cases of safety. Once we combine these hidden neurons, we get the final prediction:

$$y = \text{sign}(h_1 + h_2)$$

This shows how hidden units capture nonlinear relationships that a linear classifier cannot.

Smooth Activation: From Step to Sigmoid: Since $I(\cdot)$ is non-differentiable, we replace it with a *sigmoid* function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This makes the network differentiable and trainable via gradient descent. Note that the gradient of a sigmoid function is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Sigmoid acts like a “soft” version of a step function, gradually transitioning from 0 to 1.

2.2 Neural Networks as Function Composition

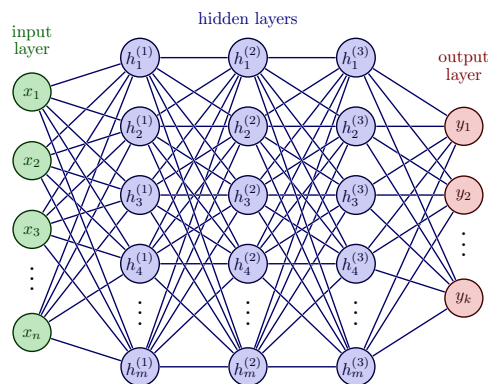


Figure 2: A neural network

A neural network is a composition of functions:

$$\hat{\mathbf{y}} = f(\mathbf{x}) = f^{(L)}(f^{(L-1)}(\dots f^{(1)}(\mathbf{x})))$$

This composition of functions is usually illustrated as a diagram in Figure 2 where green circles indicate input layer, purple circles indicate hidden layers, and red circles indicate output layer. The number of neurons in the input layer is determined by the number of features for each sample; The number of neurons in the output layer is determined by the number of classes. The number of hidden layers and the number of neurons per layer are hyperparameters that need to be determined by the engineer. Each hidden layer performs the following:

1. Linear transformation: $\mathbf{z}^{[l]} = W^{[l]}\mathbf{h}^{[l-1]} + \mathbf{b}^{[l]}$
2. Activation function: $\mathbf{h}^{[l]} = g(\mathbf{z}^{[l]})$

where $\mathbf{h}^{[0]} = \mathbf{x}$, and $g(\cdot)$ is a nonlinear activation. We can choose different $g(\cdot)$ for different layers. For instance, as output layer, if we have a multi-class classification task, the activation function can be a softmax function. In the hidden layers, relu function is typically the default choice. Different choices of activations can be found in Table 2. Note: if the activation functions of all the hidden unites in a network are taken to be linear, for any such network, we can always find an equivalent network without hidden units [1]. In short, it collapses into a linear model (e.g., perceptron).

2.3 Forward Propagation

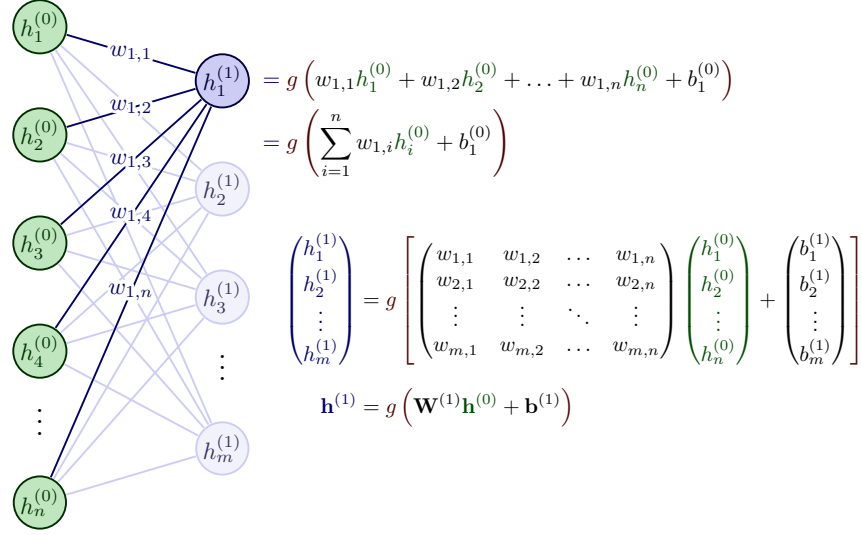


Figure 3: Illustration of feedforward propagation for one hidden neuron $h_1^{(1)}$. We put all the hidden neuron outputs in this layer as a vector $\mathbf{h}^{(1)}$. The first layer is the input layer, thus $\mathbf{h}^{(0)} = \mathbf{x}$. The model parameters for the first hidden layer are $\mathbf{W}^{[1]}$ and $\mathbf{b}^{[1]}$.

If we only have one hidden layer, the entire neural network can be represented by the following equations. Given input $\mathbf{x}$, the model computes:

$$\begin{aligned}
 \text{Hidden pre-activation : } \mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} \\
 \text{Hidden activation : } \mathbf{h}^{[1]} &= g(\mathbf{z}^{[1]}) \\
 \text{Output pre-activation : } \mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{h}^{[1]} + \mathbf{b}^{[2]} \\
 \text{Output : } \hat{\mathbf{y}} &= g(\mathbf{z}^{[2]})
 \end{aligned}$$

This process is simply evaluating the network's function. We can view neural networks as a general class of parametric nonlinear functions mapping from a vector $\mathbf{x}$ of input variables to a vector $\mathbf{y}$ of output variables.

2.4 Backpropagation

Now consider the case of binary classification in which we have a single target variable y such that $y = 1$ denotes class C_1 and $y = 0$ denotes class C_2 . We consider a network having a single output whose activation function is a logistic sigmoid to model this target variable:

$$\hat{y} = \sigma(a) = \frac{1}{1 + \exp(-1)}.$$

It represents the probability of the sample belonging to class C_1 . In the binary case, the probability of this sample belonging to class C_2 is $1 - y$. If we consider a training set of independent observations, the error/loss function, which is given by the negative log likelihood, is then a cross-entropy error function of the form;

$$L(\mathbf{w}) = - \sum_{n=1}^N \{y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n)\} \quad (1)$$

where $\hat{y}_n$ denotes $\hat{y}_n(\mathbf{x}_n, \mathbf{w})$. The objective of learning a neural network is to minimize the loss between predicted $\hat{\mathbf{y}}$ and true $\mathbf{y}$. The method to optimize the parameters (e.g., $\mathbf{W}^{[1]}$, $\mathbf{b}^{[1]}$, $\mathbf{W}^{[2]}$, $\mathbf{b}^{[2]}$) in a network is *Gradient Descent*.

In a neural network, *backpropagation* is the technique to efficiently compute gradients via the chain rule. The key idea includes three steps: (1) Compute the prediction $\hat{\mathbf{y}}$; (2) Compute the loss L ; (3) Propagate gradients backward layer-by-layer. Without backpropagation, computing gradients would require recalculating partial derivatives repeatedly. Backpropagation reuses values computed in the forward pass.

Table 2: Activation Function Examples

Function	Formula	Range	Notes
Sigmoid	$g(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	Saturates at both ends
Tanh	$g(x) = 2\sigma(2x) - 1$	$(-1, 1)$	Zero-centered
ReLU	$g(x) = \max(0, x)$	$[0, \infty)$	Efficient and widely used
Leaky ReLU	$g(x) = \max(\alpha x, x)$	$(-\infty, \infty)$	Fixes dying ReLU problem
ELU	$g(x) = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$	$(-\alpha, \infty)$	Smooth alternative
Maxout	$g(x) = \max(w_1^T x + b_1, w_2^T x + b_2)$	$(-\infty, \infty)$	Generalized linear unit

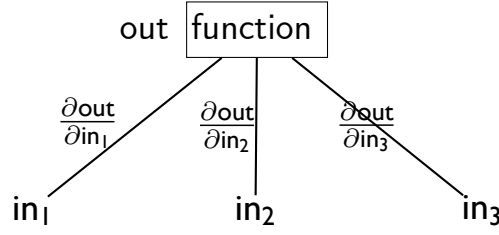


Figure 4: Computation graph for one node with multiple inputs

Mathematically, backpropagation is just grinding through the chain rule. We can visualize the computation using a computation graph as shown in Figure 4. This will avoid long equations and reveal structure of computations (modularity, efficiency, dependencies).

Partial derivatives (gradients) evaluate how much the output change if an input changes. For instance, if we have a function $out = 2in_1 + in_2in_3$, we evaluate how much the change in input in_2 would affect the output by the following: $2in_1 + (in_2 + \epsilon)in_3 = out + \epsilon in_3$ where in_3 is the partial derivative (or gradient) of in_2 .

When we have compositional functions, we apply chain rule by the following figure 5:

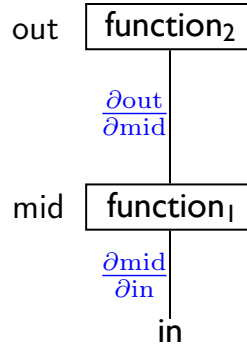


Figure 5: Computation graph for a composing function

where we get

$$\frac{\partial out}{\partial in} = \frac{\partial out}{\partial mid} \frac{\partial mid}{\partial in}$$

The goal of training a neural network is to learn the weights so that the loss (error) is minimized. Backpropagation provides an efficient procedure to compute derivatives.

Suppose we have a neural network with only one hidden layer. Given one example $(\mathbf{x}, y)$, we first operate **forward propagation**:

$$\begin{aligned}
\mathbf{z}^{[1]} &= W^{[1]} \mathbf{x} + b^{[1]} \quad (\text{First layer parameters : } W^{[1]} \text{ and } b^{[1]}) \\
\mathbf{h}^{[1]} &= \sigma(\mathbf{z}^{[1]}) \\
z^{[2]} &= W^{[2]} \mathbf{h}^{[1]} + b^{[2]} \quad (\text{Last layer parameters : } W^{[2]} \text{ and } b^{[2]}) \\
\hat{y} &= \sigma(z^{[2]})
\end{aligned}$$

Then we calculate the error or loss on one example:

$$L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Then we apply **back propagation** to calculate the gradients of parameters. Because the output uses a sigmoid, with a cross-entropy loss, a very clean derivative appears:

$$\frac{\partial L}{\partial z^{[2]}} = \hat{y} - y$$

The gradients of the output layer parameters follow:

$$\frac{\partial L}{\partial W^{[2]}} = \frac{\partial L}{\partial z^{[2]}} h^{[1]\top}, \quad \frac{\partial L}{\partial b^{[2]}} = \frac{\partial L}{\partial z^{[2]}}$$

Using the chain rule, the gradient flowing into the hidden layer is

$$\begin{aligned}
\frac{\partial L}{\partial z^{[1]}} &= (W^{[2]\top} \frac{\partial L}{\partial z^{[2]}}) \circ h^{[1]} \circ (1 - h^{[1]}) \\
\frac{\partial L}{\partial W^{[1]}} &= \frac{\partial L}{\partial z^{[1]}} \mathbf{x}^\top, \quad \frac{\partial L}{\partial b^{[1]}} = \frac{\partial L}{\partial z^{[1]}}
\end{aligned}$$

where “ $\circ$ ” denotes elementwise multiplication, and

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

After we get all the gradients, we apply gradient descent updates:

$$\begin{aligned}
W^{[1]} &= W^{[1]} - \eta \frac{\partial L}{\partial W^{[1]}} \\
\mathbf{b}^{[1]} &= \mathbf{b}^{[1]} - \eta \frac{\partial L}{\partial \mathbf{b}^{[1]}} \\
W^{[2]} &= W^{[2]} - \eta \frac{\partial L}{\partial W^{[2]}} \\
b^{[2]} &= b^{[2]} - \eta \frac{\partial L}{\partial b^{[2]}}
\end{aligned}$$

2.5 Vectorization Across Batches

Using matrix operations instead of loops accelerates computation, especially on GPUs. For N training examples, forward pass becomes:

$$\begin{aligned}
Z^{[1]} &= W^{[1]} X + b^{[1]} \\
H^{[1]} &= g(Z^{[1]}) \\
Z^{[2]} &= W^{[2]} H^{[1]} + b^{[2]} \\
H^{[2]} &= g(Z^{[2]})
\end{aligned}$$

where $X \in \mathbb{R}^{d \times N}$ contains all batch inputs. Gradients are averaged over the batch size N for efficiency. This approach fully leverages hardware acceleration and reduces overhead.

References

- [1] C. M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006.
- [2] R. Zemel, R. Urtasun, and S. Fidler. Lecture 10: Neural networks. https://www.cs.toronto.edu/~urtasun/courses/CSC411_Fall16/10_nn1.pdf.